Sakshar — Project Overview

At a glance

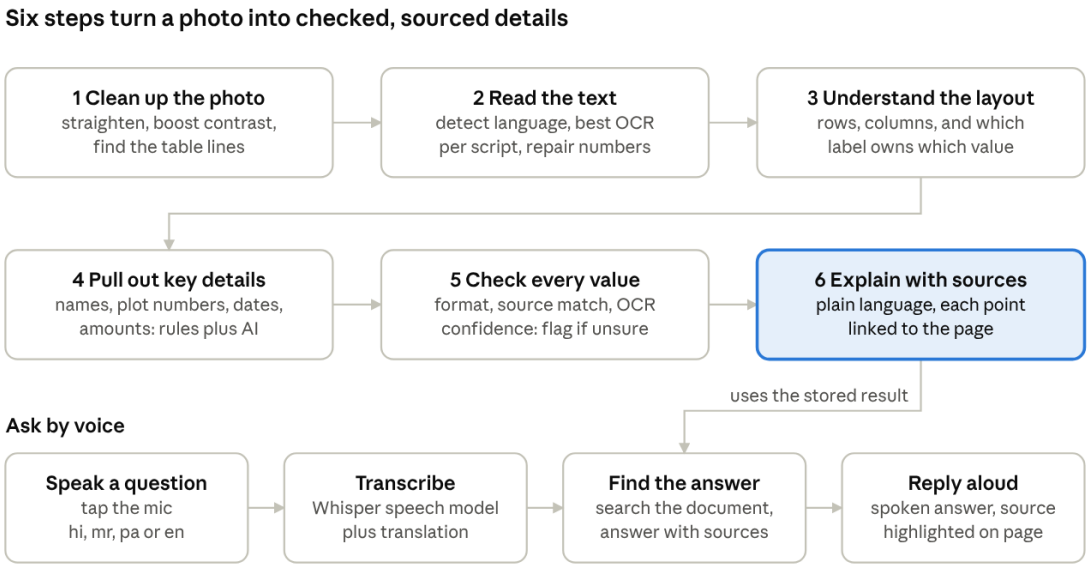
Sakshar is a working web application that reads Indian land records and government forms from a photo or PDF. It pulls out the key details, flags anything it is unsure of, explains the document in plain Hindi, Marathi, Punjabi or English, and answers spoken questions about it. It is complete as a prototype: the full feature set runs end to end on one laptop, 50 automated tests pass, and a live end-to-end check passed every feature.

The problem it solves. Land records (Khatauni, 7/12 Satbara, Jamabandi) and official forms are written in dense administrative language that many owners cannot easily read. Misreading one number, such as a plot number, an area or a loan entry, can cost someone their land rights or a benefit. Sakshar makes these documents understandable without asking the user to trust a black box. Every statement it makes points to the exact text on the page, and uncertain values are flagged for a human to check.

Key fact	Value
Languages	Hindi, Marathi, Punjabi, English (reading, explaining, speaking)
Documents	UP Khatauni, Maharashtra 7/12, Punjab Jamabandi, application forms, certificates, plus a generic fallback
Input	Phone photo, scan or PDF, up to 20 MB and 6 pages
Output	Verified fields, a plain-language explanation with sources, spoken Q&A
Stack	Python, FastAPI, OpenCV, Tesseract, PaddleOCR, OpenAI API, Whisper, IndicTrans2, Parler-TTS
Hardware	One laptop: CPU for reading documents; a 6 GB NVIDIA GPU for voice
Code	~5,500 lines (3,800 of them application Python) in 66 files, github.com/iwantacrepe/Sakshar
Status	Prototype complete: every feature works end to end, 93% field accuracy on the test set, 50 automated tests

How it works

Every document goes through six automatic steps, ending in an explanation whose every point links back to the page. Questions, spoken or typed, are answered from that stored result, so the image is processed only once.



Sakshar pipeline · 6 document steps, 4 voice steps

Between steps 5 and 6, the person sees any flagged values and can confirm or correct them; the explanation then regenerates with the corrected values. Steps 1–5 and the voice models run on the local machine; the explanation and answer text come from the AI service.

Worked example: a Hindi Khatauni, start to finish

This follows one real run of the Hindi sample, a UP land record photographed slightly crooked, with a purple stamp and one smudged digit. It took about 7 seconds from upload to verified fields. Each step below shows what the system actually produced.

1. Upload. The user drops `hi_khatauni.png` on the home page and picks "Explain in: Same as document". The page switches to a live progress list that ticks through the stages as they finish.

2. Clean up the image. The photo is tilted. Sakshar measures the tilt from the table's printed lines: **+3.5°**, which matches the 3.5° it was rotated by. It straightens the colour image, so the stamp stays purple in the viewer, and builds two cleaned copies, one for each OCR engine.

3. Work out the language. Unicode counts and marker words identify **Hindi, Devanagari script**. For example, खसरा and तहसील point to Hindi, while आहे would have pointed to Marathi.

4. Read the text. Tesseract's Hindi model reads the page, and an English-only pass corrects the numbers it misreads. Before the correction, `00127` came out as `0027` and `214` was lost; after it, both are right. PaddleOCR finds any lines Tesseract skipped. Result: **31 text blocks**, each with a confidence score.

5. Understand the layout. The printed table lines become rows and columns. The header "खसरा संख्या" (plot number) is linked to the cell beneath it, "214". The two-line owner cell, "राम प्रसाद पुत्र श्याम लाल / निवासी रामपुर कलां", stays together as one value.

6. Identify the document. Keywords like खतौनी, खसरा and फसली classify it as a **UP Khatauni**, with 93% confidence.

7. Extract the key details. Rules for the Khatauni schema, cross-checked by the LLM, give:

Field	Value	Status
Owners	राम प्रसाद …; सीता देवी …	OK
Father / husband	श्याम लाल; राम प्रसाद	OK
Khata number	00127	OK
Khasra (plot) number	214	OK
Area	… हेक्टेयर	**Flagged:** smudged digit, OCR confidence 48%
Village	रामपुर कलां	**Flagged:** OCR confidence 74% (threshold 75%)
Tehsil / District	नगीना / बिजनौर	OK
Mutation order date	12/03/2023	OK
Land revenue	₹45.00	OK
Aadhaar	XXXX-XXXX-9005	OK: checksum valid; masked before anything leaves the machine

8. Verify. Each value is checked against four tests:

- Does it parse as a date, amount or area?
- Is it actually present in the text it came from?
- Was the OCR confident?
- Do the two extractors agree?

The two flagged fields show on screen as amber cards with a crop of the original image, so the person can confirm or correct them.

9. Explain. The LLM writes 4–10 short points in Hindi. Each cites the text blocks it came from, for example "खसरा संख्या 214 है।", tagged with the id of the block holding "214". A checker then confirms that every number in each point appears in the cited text. Clicking a point highlights its source on the page image.

10. Ask by voice. The user taps the mic and asks "मेरी ज़मीन का खसरा नंबर क्या है?" ("What is my land's khasra number?"):

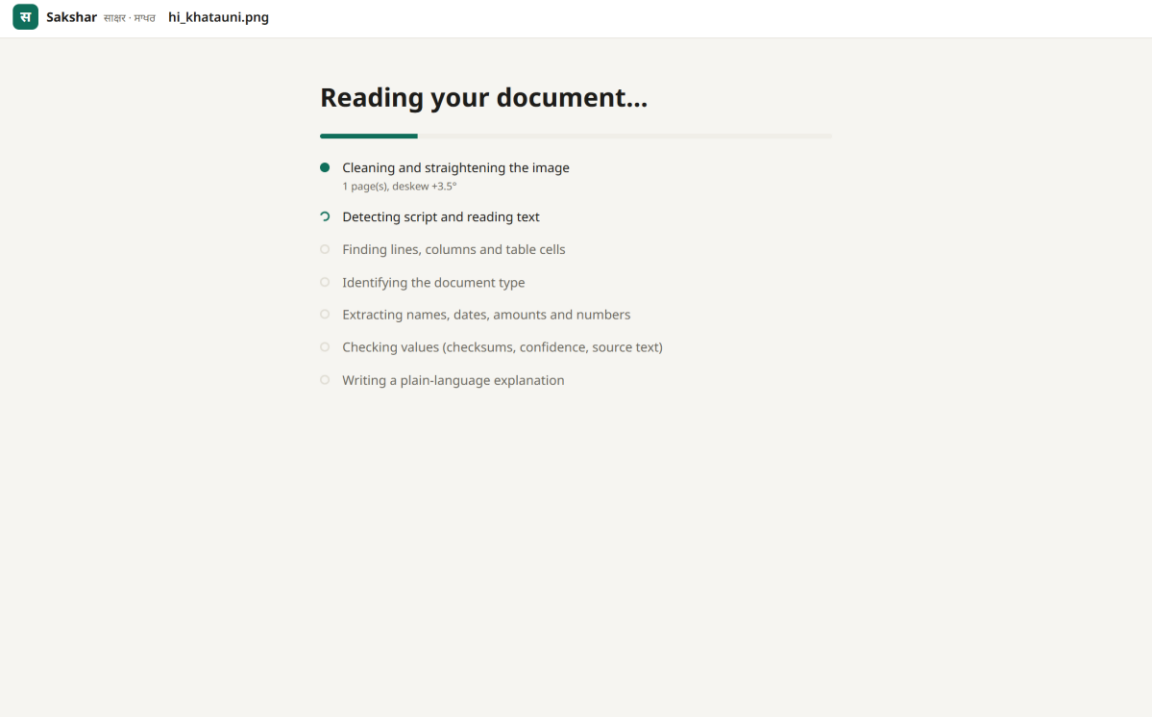
1. Whisper transcribes it.

2. IndicTrans2 translates it as "What is the **Khasra** number of my land?". Without a fix, "खसरा" was first translated as "measles", its other meaning in Hindi.
3. Sakshar finds the answer in the document.
4. It replies aloud: "आपकी खसरा संख्या दो एक चार है…" ("Your khasra number is two-one-four…"), with the plot number spelled digit by digit, and highlights the source on the page.

Screenshots from a local run

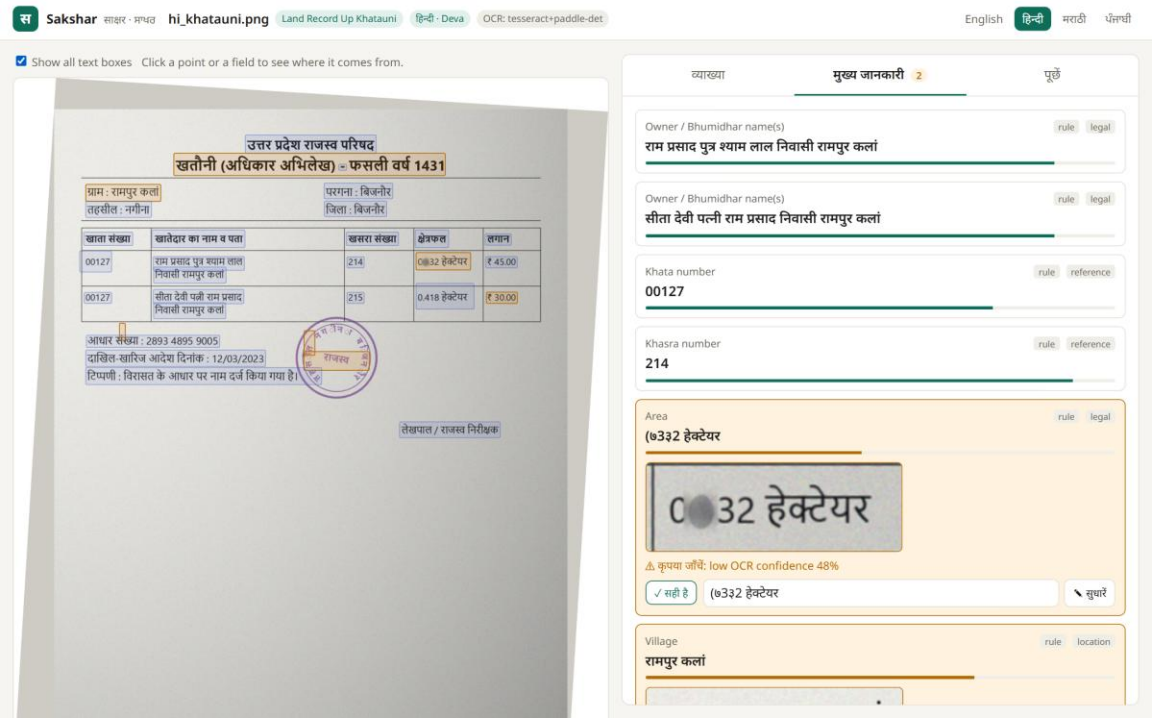
Every image below comes from the app running locally in Microsoft Edge on the bundled sample documents, captured with Playwright. Nothing was mocked up.

Upload and live progress. The person drops in a photo; each stage ticks off as it finishes. The first stage has already measured and corrected the photo's tilt (+3.5°).

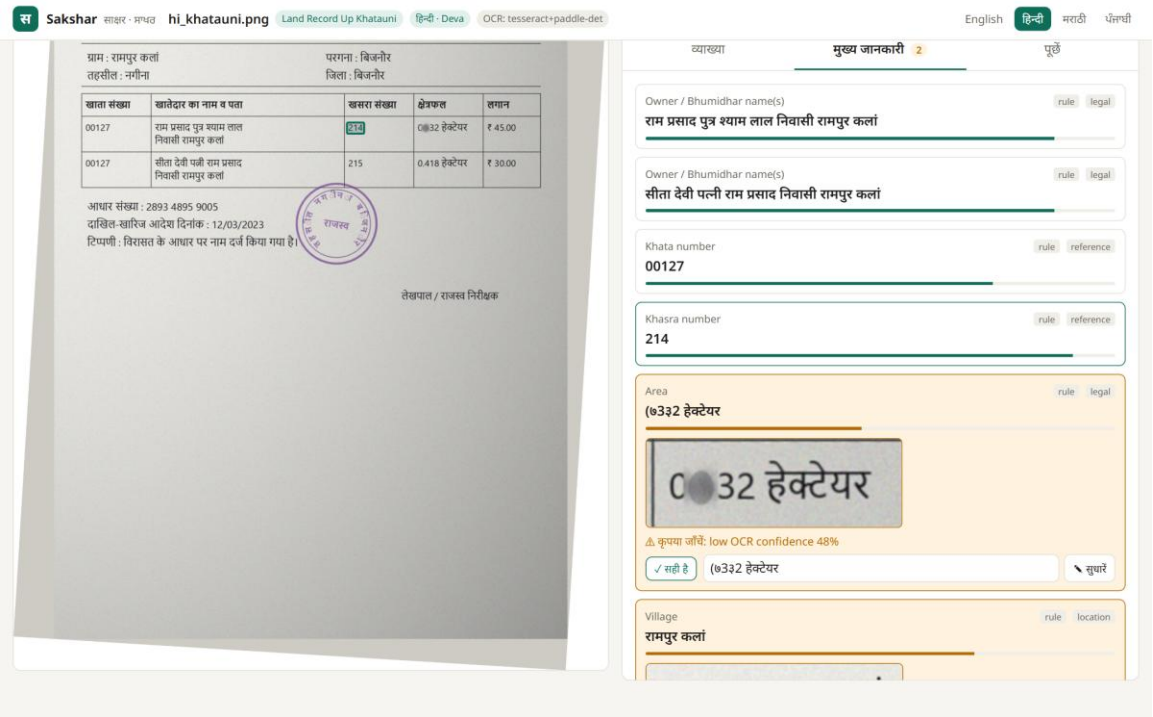


Reading a Hindi Khatauni.

- **Left:** the straightened page, with every recognised text block outlined and low-confidence text outlined in amber.
- **Right:** the extracted details in a Hindi interface.
- **Header chips:** the detected document type, language and script, and the OCR route used.



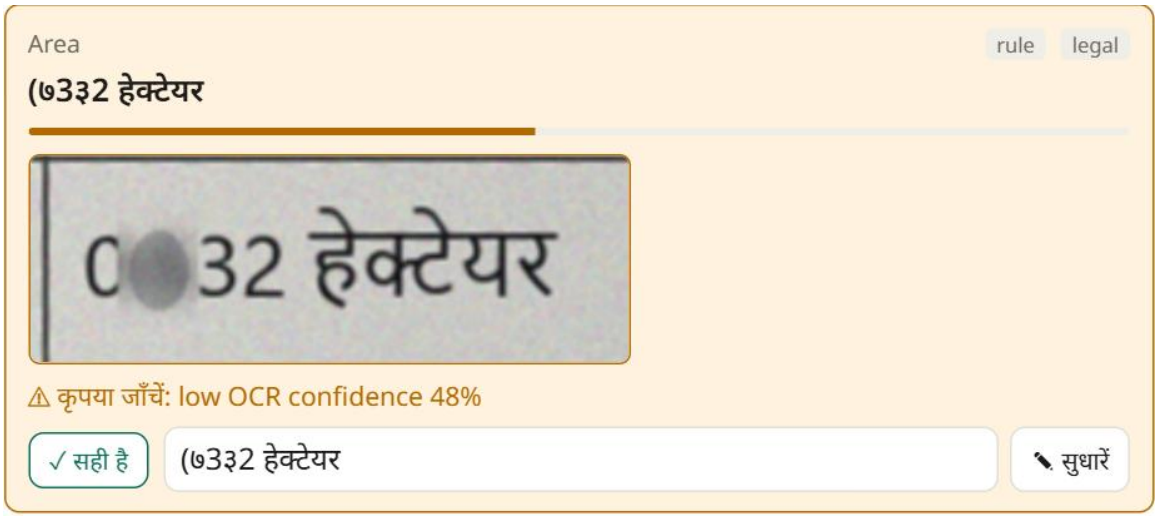
Every value points to its source. Clicking "Khasra number 214" highlights the exact table cell it was read from.



Uncertain values are flagged, not guessed. The area value contains a deliberately smudged digit. Sakshar shows:

- a crop of the original;

- the reason ("low OCR confidence 48%");
- *Looks right* / *Correct it* buttons.



After correction, the value is marked as entered by the person and normalized to 0.632 hectare.

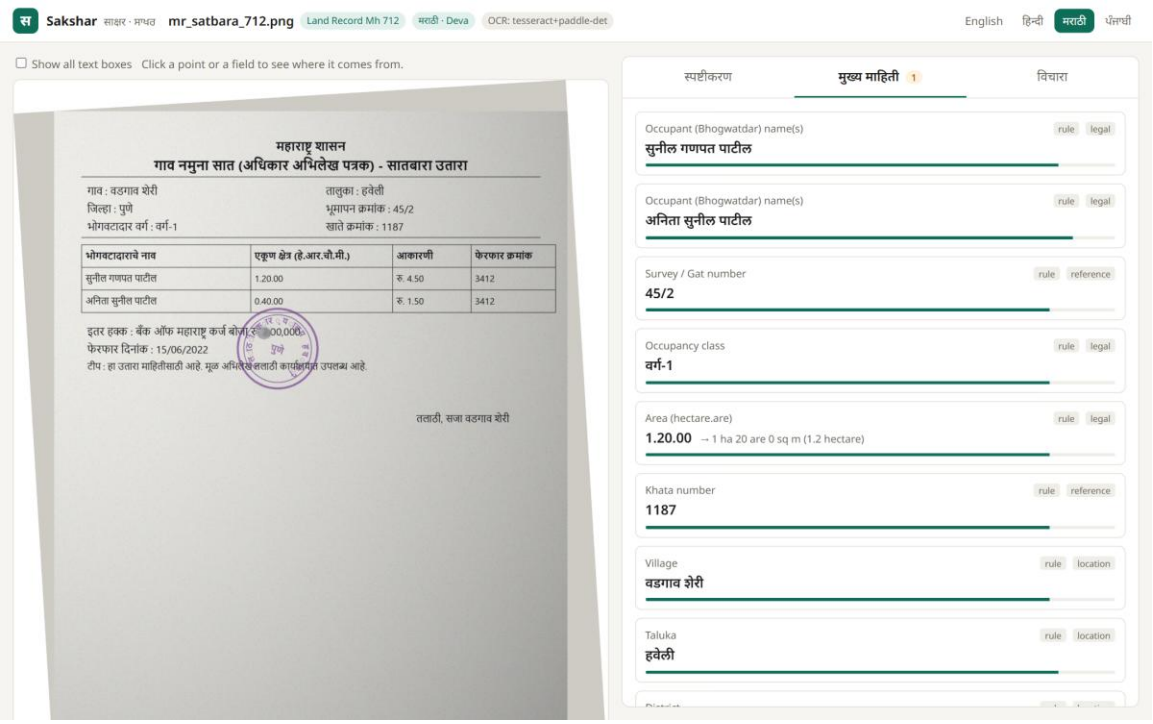


Asking by voice. This question was spoken into the browser's microphone in Hindi. Whisper transcribed it correctly, including the plot number read out digit by digit ("दो एक चार").

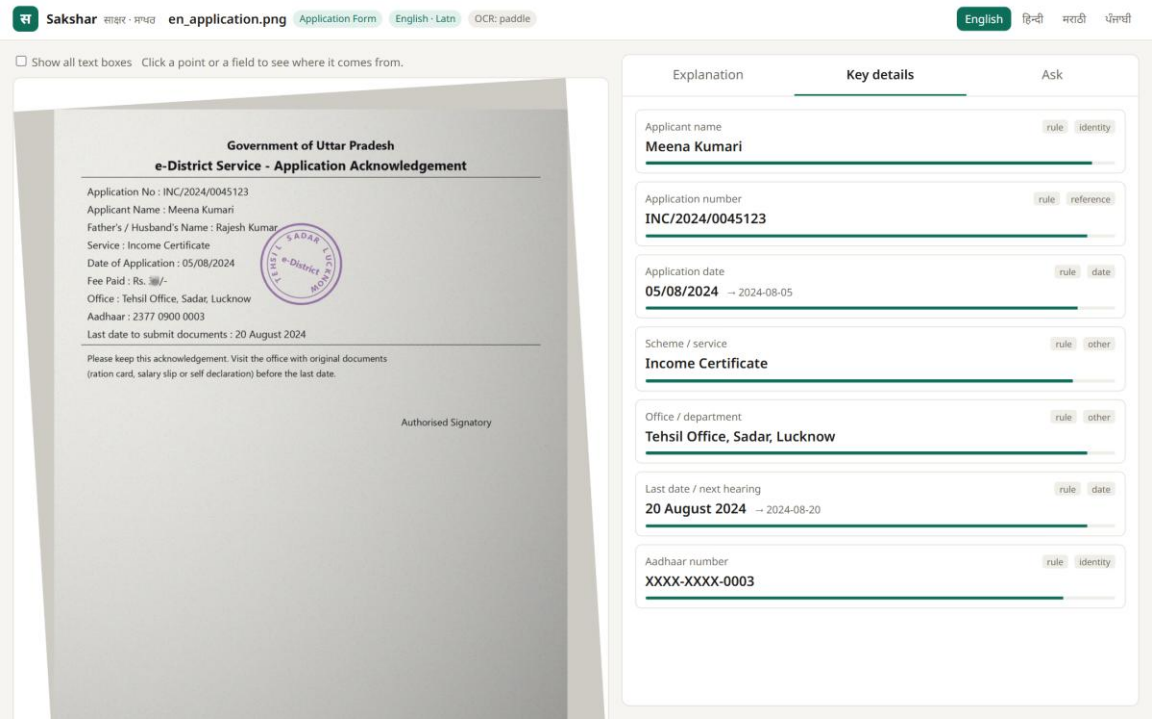
आपकी खसरा संख्या दो एक चार है और जमी जांपुर गावे हैं।

Marathi, Punjabi and English documents.

- **Marathi 7/12:** the area `1.20.00` is understood as 1 hectare 20 are.
- **Punjabi Jamabandi:** owner and cultivator cells, read with lower confidence, carry review crops.
- **English application:** every field is read correctly, and the Aadhaar number is masked.



[image: Punjabi Jamabandi]



Dark theme and phone layout.

[image: Dark theme]

hi_khatauni.png

स Sakshar

Land Record Up Khatauni

हिन्दी · Deva

English | हिन्दी | मराठी | ਪੰਜਾਬੀ

Owner / Bhumidhar name(s) rule legal

सीता देवी पत्नी राम प्रसाद निवासी रामपुर कलां

Khata number rule reference

00127

Khasra number rule reference

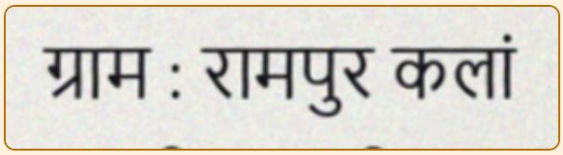
214

Area you legal

0.632 हेक्टेयर → 0.632 hectare

Village rule location

रामपुर कलां



⚠ कृपया जाँचें: low OCR confidence 74%

✓ सही है | रामपुर कलां | ✎ सुधारें

Tehsil rule location

नगीना

Why the answers can be trusted

The core design principle: **the system never states a fact it cannot point to on the page.** Five safeguards enforce this, and each is covered by automated tests.

Safeguard	What it does	Why it matters
Source citations	Every explanation point and every answer carries the ids of the text blocks it relies on; clicking it highlights them on the image	The user can check any claim in one click
Grounding check	Code, not the AI, checks that every number, date and amount in a statement appears in its cited text. A wrong citation is repaired; a value found nowhere marks the statement "Not verified"; an unsupported numeric claim is dropped	Stops the AI from inventing or garbling plot numbers, areas or amounts
Review flags	A field is flagged when OCR confidence is low, a validator fails (bad date, impossible amount, Aadhaar checksum), the value isn't in its source, or two extractors disagree. Legal and financial fields have a stricter bar (85% vs 75%)	Uncertainty is shown to the person instead of hidden; they confirm or correct, and the explanation regenerates
Two independent extractors	Rule-based extraction and AI extraction run separately; agreement is recorded, disagreement is flagged	One method catches the other's mistakes
Privacy by default	Aadhaar numbers are validated locally and masked (`XXXX-XXXX-1234`) **before** any text is sent to the AI service or saved; raw provider error text is never shown in the UI	Sensitive IDs never leave the machine or sit on disk

In practice: on all four sample documents, every deliberately smudged value was flagged, and no correctly read value failed validation.

Technology choices

Each component was chosen, and in several cases changed, because of how it measured on the actual documents and hardware.

Area	Choice	Reason
Web app	Python FastAPI + server-rendered pages (HTMX)	One language for the whole project; live progress via server-sent events; no separate front-end build
Image cleanup	OpenCV	Straightening, contrast and table-line detection
Reading Hindi / Marathi / Punjabi	Tesseract 5	Clearly the most accurate on printed Devanagari and Gurmukhi in head-to-head tests. PaddleOCR's Devanagari model dropped spaces and reported

Area	Choice	Reason
		false high confidence; Paddle has no Punjabi model
Reading English, finding text lines	PaddleOCR 3 (mobile models)	Strong on English; its line detector finds text Tesseract misses. The mobile models cut processing from ~50 s to 8 s per page
Explanations and answers	OpenAI API (`gpt-4o-mini`, configurable) with structured outputs	Reliable JSON with citations. Any OpenAI-compatible endpoint, including a self-hosted one, can be dropped in
Speech to text	Whisper large-v3-turbo	The smaller Whisper model produced unusable Punjabi; turbo transcribes all three languages well
Translation	IndicTrans2 (AI4Bharat)	Purpose-built for Indian languages, runs locally
Text to speech	Indic Parler-TTS (AI4Bharat)	Natural Hindi, Marathi and Punjabi voices, runs locally
Storage	SQLite	Zero setup; enough for a single-machine deployment
Environment	Conda (conda-forge), Python 3.11	Reproducible install of a heavy ML stack on Windows; version pins resolve real conflicts between the OCR and voice libraries

Everything except the explanation and answer calls runs on the local machine.

Quality and testing status

All **50 automated tests pass**, and a final live end-to-end check of the running app passed every feature listed below.

Automated tests (48) cover:

- image straightening accuracy;
- language detection;
- number, date and amount parsing in all four languages;
- the Aadhaar checksum and masking;
- table layout;
- every review rule;
- the citation checker;
- search that tolerates OCR typos;
- a full upload-to-answer flow with the AI mocked;
- one regression test for each serious bug found during development.

Live end-to-end check against the running server:

- ☒ Upload from the browser form, including 4 documents at once and a PDF

- ☒ Live progress stream through all stages
- ☒ Each document gets the correct language and type
- ☒ Document page with image, highlight overlay, extracted details and mic button
- ☒ Review: confirm and correct a flagged value; the explanation refreshes
- ☒ Voice questions in Hindi, Marathi and Punjabi transcribed correctly from browser-format audio
- ☒ Spoken answers ("Listen") in all three languages
- ☒ Typed questions
- ☒ Bad files, empty audio and unknown documents rejected cleanly
- ☒ App works offline apart from the AI calls (front-end libraries bundled)

Test documents. The test set has four records, one per language: a UP Khatauni, a Maharashtra 7/12, a Punjab Jamabandi and an English application. Each was given real-world damage: a 3.5° tilt, an ink stamp over a date, uneven lighting, noise and one smudged digit.

Measured extraction accuracy. Every field on the test documents has a known correct value, so each extracted value is checked against it.

Measure	Result
Values extracted correctly	**41 of 44 (93% precision)**
Values on the page found	41 of 49 (84% recall)
Wrong values flagged for review	**3 of 3 (100%)**: every error came from a smudged digit, and none was shown as certain
Language and document type detected	4 of 4

OCR engines, preprocessing and page-segmentation modes were also compared head to head on the same documents; that comparison decided which engine reads each script.

Performance and hardware

A one-page document goes from upload to verified details in **7–20 seconds** on a laptop CPU. A voice question is transcribed in **2–3 seconds**. The spoken reply is the slowest step, at **8–25 seconds**.

Step	Typical time
Read and verify a Hindi or Marathi page	7–9 s
Read and verify a Punjabi page (more re-reads)	13–20 s
Read and verify an English page	8 s
Transcribe a voice question	2–3 s
Translate a question	under 1 s

Step	Typical time
Generate a spoken reply	8–25 s
First voice request after start-up (models load)	1–1.5 min, once

Hardware. Development used a laptop with an NVIDIA RTX 3060 (6 GB). The voice models stay loaded together and use **3.7 GB** of GPU memory. Document reading runs on the CPU, so the app also works without a GPU: voice is then slower, or can be switched off.

Demo tip. Ask one warm-up question after starting the server, so the first live question is fast.

Key engineering challenges solved

Thirty-two problems surfaced while running the system on real models and images; all are fixed, and the detailed log is in the Technical Design & Engineering Reference. These eight had the biggest effect on quality or reliability:

Challenge	Impact if unsolved	Solution
The obvious OCR model for Hindi was confidently wrong (no spaces, broken letters, 95% confidence)	Garbled text the system couldn't even flag	Measured both engines head to head; switched Indian scripts to Tesseract and kept PaddleOCR for line finding and English
Hindi and Marathi OCR misread ordinary numbers (`00127` → `0027`, `214` lost)	Wrong plot and account numbers, the most critical values	A second number-only reading pass that repairs digits without touching words
Table labels sit in headers, values in cells below	Fields filled with the wrong numbers, e.g. Aadhaar digits as the khata number	Detected the printed table lines to reconstruct rows and columns, then paired each header with its own column
A slight leftover tilt merged neighbouring table rows	Values assigned to the wrong row	Measured skew from the table's straight lines instead of the text: now exact (3.50°) on every sample
"खसरा" (plot number) also means "measles" in Hindi; the translator chose measles	Voice questions misunderstood	Domain terms are protected before translation
Spoken numbers came out unintelligible	Users couldn't hear their plot number	Numbers are spelled digit by digit in the target language
Punjabi speech recognition returned gibberish with the small model	Punjabi voice unusable	Upgraded to Whisper large-v3-turbo with a script-correction step

Challenge	Impact if unsolved	Solution
Server-only crashes: the OCR library failed under concurrency, and the voice library failed after OCR had loaded	App failures that standalone testing missed	All OCR-library work runs on one dedicated thread; libraries load in a safe order. Both are covered by regression tests

Risks, limitations and next steps

As a college prototype, Sakshar has been evaluated on a small test set built to mimic real scans. A larger evaluation on scanned records from each state would be the natural next step before any deployment.

Risk / limitation	Effect	Mitigation today
Evaluated on a small test set	Accuracy on other states' layouts and on court orders or letters is not yet measured	Review flags catch low-confidence values; the AI extractor cross-checks the rule-based one
Handwriting not supported	Older handwritten registers will read poorly	Low OCR confidence triggers review
A stamp fully covering a value	That field may be missing rather than flagged	Missing *required* fields are reported in the progress view
Speech auto-detection mistakes Marathi and Punjabi for Hindi	Wrong transcript if the language isn't set	The app uses the language selected on the page
Spoken replies take 8–25 s	Slower conversation	Audio is cached; text answer shows first
Single-machine design, no login	Not ready for multi-user hosting	Intended for local or help-desk use
Explanations depend on an external AI service	Needs internet and an API key	Any compatible endpoint, including self-hosted, can be configured; all other features work without it